

1. What is Python and why is it popular?

Python is a high-level, interpreted programming language celebrated for its simplicity, readability, and broad applicability across different domains like web development, data analysis, AI, and more.

Here are several reasons why Python has gained widespread popularity:

- i. Simplicity and Readability:
- ii. Versatility
- iii. Large Ecosystem
- iv. Community Support
- v. Cross-platform Compatibility
- vi. Ease of Learning
- vii. Scalability
- viii. Interpreted Language

2. How do you install Python on your computer?

Installing Python on your computer is a straightforward process. Here are the general steps:

- i. Download Python: Visit the official Python website at <https://www.python.org/>. Navigate to the Downloads section and choose the version of Python you want to install.
- ii. Run the Installer: Once the download is complete, run the installer file you downloaded. On Windows, this typically has a filename like `python-3.x.x.exe`, where `3.x.x` represents the version number. On macOS, it may have a `.pkg` extension. On Linux, you might install Python through your distribution's package manager.
- iii. Configure the Installation: During the installation process, you may have the option to customize the installation settings. Make sure to check the box that adds Python to your system PATH.
- iv. Verify the Installation: After the installation is complete, you can verify that Python is installed correctly by opening a command prompt (Windows) or terminal (macOS/Linux) and typing `python --version` or `python3 --version` depending on your system.
- v. Optional: Install a Code Editor or IDE: While Python can be run from the command line, many developers prefer using Integrated Development Environments (IDEs) or code editors for writing and debugging code.

3. What version of Python are you currently using, and why did you choose it?

Currently using Python 3.10. The primary reasons for choosing this version are its improved performance and the new features it offers. Python 3.10 introduces several enhancements, including:

- Structural Pattern Matching: This feature, often compared to switch/case statements found in other languages, allows for more flexible and readable conditional statements.
- Precise Types: Enhancements in type hinting make code easier to debug and maintain, especially in larger codebases.
- Parenthesized Context Managers: This improvement allows multiple context managers in a single **with** statement to be enclosed in parentheses, enhancing readability.
- Performance Improvements:

Python 3.10 includes optimizations that improve the performance of Python applications. For example, the interpreter now executes faster, benefiting all Python programs.

These features make Python 3.10 a robust choice for developing scalable and efficient applications. Moreover, staying updated with the latest version ensures compatibility with new libraries and frameworks, providing access to the latest tools and features for development.

4. How do you write a basic Python program?

Write code in a

`.py` file and execute it using `python filename.py` in a command prompt or terminal.

```
1 # This is a basic Python program
2 print("Hello, world!")
```

- Save the file with a `.py` extension, for example, `hello.py`.
- Open a command prompt (Windows) or terminal (macOS/Linux).
- Navigate to the directory where you saved the `hello.py` file using the `cd` command.
- Once you're in the correct directory, type

`python hello.py` (or `python3 hello.py` on some systems) and press Enter.

You should see the output:

```
1 Hello, world!
```

5. What are Python's key features and strengths?

Python boasts several key features and strengths that contribute to its popularity among developers:

- Readable and Simple Syntax
- Versatility
- Large Ecosystem
- Community Support
- Cross-platform Compatibility
- Ease of Learning
- Interpreted Language
- Scalability

6. What are variables in Python?

Variables are symbols that you assign data values to; they are dynamically typed, meaning the data type is inferred at runtime.

```
2 name = "John" # A string variable
3 age = 30      # An integer variable
4 height = 5.9  # A float variable
5 is_student = True # A boolean variable
6
7 # Printing the values of variables
8 print(name)
9 print(age)
10 print(height)
11 print(is_student)
```

PYTHON



Expand

7. How do you comment code in Python?

There are two ways to add comments

- Single-line Comments:

Single-line comments begin with the

`#` character and continue until the end of the line. Anything after `#` on the same line is considered a comment.

```
1 # This is a single-line comment
```

```
2 print("Hello, world!") # This is also a comment
```

ii. Multi-line Comments (Docstrings):

Multi-line comments can be enclosed within triple quotes (

""" or '''). While they are not strictly comments, they are commonly used as documentation strings, or "docstrings", to provide documentation for functions, modules, classes, or methods.

PYTHON

```
1 """
2 This is a multi-line comment or docstring.
3 It can span multiple lines.
4 """
5
6 '''
7 This is also a multi-line comment or docstring.
8 '''
```

8. What are the basic data types in Python?

i. Integer (int)

ii. Floating-point (float)

iii. String (str)

iv. Boolean (bool)

v. NoneType (None)

vi. List (list)

vii. Tuple (tuple)

viii. Dictionary (dict)

ix. Set (set)

9. How do you perform arithmetic operations in Python?

Use operators like

+ , - , * , / , // , % , and ** to perform calculations. Use parentheses to define operation order.

i. Addition (+):

Adds two operands.

PYTHON

```
1 result = 5 + 3
2 print(result) # Output: 8
```

ii. Subtraction (-):

Subtracts the second operand from the first.

PYTHON

```
1 result = 10 - 7
2 print(result) # Output: 3
```

iii. Multiplication (*):

Multiplies two operands.

PYTHON

```
1 result = 4 * 6
2 print(result) # Output: 24
```

iv. Division (/):

Divides the first operand by the second. Always returns a floating-point number.

PYTHON

```
1 result = 15 / 4
2 print(result) # Output: 3.75
```

v. Floor Division (//):

Divides the first operand by the second and returns the integer part of the result (floor division).

PYTHON

```
1 result = 15 // 4
2 print(result) # Output: 3
```

vi. Modulus (%):

Returns the remainder of the division of the first operand by the second.

PYTHON

```
1 result = 15 % 4
2 print(result) # Output: 3
```

vii. Exponentiation (**):

Raises the first operand to the power of the second.

PYTHON

```
1 result = 2 ** 3
2 print(result) # Output: 8
```

10. What is type conversion in Python?

Use functions like

`int()` , `float()` , `str()` , `bool()` , `list()` , `tuple()` , `set()` , and `dict()` to convert data types.

i. `int()`:

Converts a value to an integer data type.

PYTHON

```
1 x = int(3.14)
2 print(x) # Output: 3
```

ii. `float()`:

Converts a value to a floating-point data type.

PYTHON

```
1 x = float("5.6")
2 print(x) # Output: 5.6
```

iii. `str()`:

Converts a value to a string data type.

PYTHON

```
1 x = str(42)
2 print(x) # Output: "42"
```

iv. bool:

Converts a value to a boolean data type.

PYTHON

```
1 x = bool(0)
2 print(x) # Output: False
```

v. list:

Converts a sequence (e.g., tuple, string) to a list.

PYTHON

```
1 x = list((1, 2, 3))
2 print(x) # Output: [1, 2, 3]
```

vi. tuple:

Converts a sequence (e.g., list, string) to a tuple.

PYTHON

```
1 x = tuple([1, 2, 3])
2 print(x) # Output: (1, 2, 3)
```

vii. set():

Converts a sequence (e.g., list, tuple, string) to a set.

PYTHON

```
1 x = set([1, 2, 3, 3]) # Duplicate elements are automatically removed
2 print(x) # Output: {1, 2, 3}
```

viii. dict():

Converts a sequence of key-value pairs (e.g., list of tuples) to a dictionary.

PYTHON

```
1 x = dict([(1, 'one'), (2, 'two'), (3, 'three')])
2 print(x) # Output: {1: 'one', 2: 'two', 3: 'three'}
```

11. How do you take input from the user in Python?

Use the

`input()` function to capture user input as a string.

```
1 # Prompt the user to enter their name
2 name = input("Enter your name: ")
3
4 # Print a greeting using the input name
5 print("Hello, " + name + "!")
6
7 # Take input and convert it to an integer
8 age = int(input("Enter your age: "))
9
```

```
10 # Print the input value
```

Expand

12. How does string formatting work in Python?

Utilize

% operator, format() method, or f-strings (f"{}") to insert variables into strings dynamically.

i. Using the % Operator (printf-style formatting):

PYTHON

```
1 name = "Alice"
2 age = 30
3 formatted_string = "Hello, %s! You are %d years old." % (name, age)
4 print(formatted_string)
```

Output:

```
1 Hello, Alice! You are 30 years old.
```

ii. Using the format() Method:

PYTHON

```
1 name = "Bob"
2 age = 25
3 formatted_string = "Hello, {}! You are {} years old.".format(name, age)
4 print(formatted_string)
```

Output:

```
1 Hello, Bob! You are 25 years old.
```

iii. Using f-strings (Formatted String Literals):

PYTHON

```
1 name = "Charlie"
2 age = 35
3 formatted_string = f"Hello, {name}! You are {age} years old."
4 print(formatted_string)
```

Output:

```
1 Hello, Charlie! You are 35 years old.
```

13. What are Python's lists and how do you use them?

Lists are mutable, ordered collections of items that support operations like indexing, appending, and removing elements.

Here's an example of how to create and use a list in Python:

```
1 # Create a list of numbers
2 numbers = [1, 2, 3, 4, 5]
3
```

```

4 # Create a list of strings
5 fruits = ["apple", "banana", "cherry"]
6
7 # Create a list of mixed data types
8 mixed = [1, "hello", 3.14, True]
9
10 # Access elements of a list using indexing

```

Expand

14. How do you access elements in a list?

Use indexing (

`list[index]`) and slicing (`list[start:end:step]`) to access and extract elements from a list.

i. Indexing: You can access individual elements in a list by specifying their index within square brackets

`[]` .

PYTHON

```

1 numbers = [1, 2, 3, 4, 5]
2 print(numbers[0]) # Output: 1
3 print(numbers[2]) # Output: 3

```

ii. Negative Indexing:

You can also use negative indices to access elements from the end of the list.

PYTHON

```

1 numbers = [1, 2, 3, 4, 5]
2 print(numbers[-1]) # Output: 5
3 print(numbers[-2]) # Output: 4

```

iii. Slicing:

Slicing is done by specifying a start index, an end index (exclusive), and an optional step size within square brackets

`[start:end:step]` .

PYTHON

```

1 numbers = [1, 2, 3, 4, 5]
2 print(numbers[1:4]) # Output: [2, 3, 4]
3 print(numbers[:3]) # Output: [1, 2, 3]
4 print(numbers[::2]) # Output: [1, 3, 5]

```

iv. Accessing Nested Lists:

If a list contains nested lists as elements, you can use multiple levels of indexing to access elements within the nested lists.

PYTHON

```

1 nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2 print(nested_list[1][2]) # Output: 6 (accessing the element at index 2 in the second nested list)

```

15. What are tuples in Python and how are they different from lists?

In Python, a tuple is a data structure similar to a list, but with one key difference: tuples are immutable, meaning they cannot be modified after creation.

The main differences between tuples and lists are:

- Mutability:** Tuples are immutable, meaning their elements cannot be modified after creation. Lists, on the other hand, are mutable, allowing for modifications to their elements.
- Syntax:** Tuples are defined using parentheses `()` with elements separated by commas, while lists are defined using square brackets `[]` .

- iii. Performance: Tuples are generally more memory-efficient and faster to access than lists, especially for large collections of data. This is because tuples are immutable and their elements are fixed at creation time, whereas lists may require resizing and reallocation of memory when modified.
- iv. Use Cases: Tuples are often used in situations where immutability is desired, such as representing fixed collections of data or as keys in dictionaries (since keys in dictionaries must be immutable). Lists are used when mutability is required or when the order of elements matters.

16. What is slicing in Python?

- Slicing in Python refers to the process of extracting a subset of elements from a sequence (such as a list, tuple, or string) based on specified indices or a range of indices.
- Slicing extracts a subset of elements from a sequence (like lists or strings) using the syntax `[start:end:step]`, where `start` is inclusive and `end` is exclusive.

17. How do you iterate over a list in Python?

Iterate over a list using a

`for` loop or a `while` loop. Iterating over a list allows you to access each element of the list one by one and perform operations on them.

- i. Using a `for` Loop:

PYTHON

```
1 numbers = [1, 2, 3, 4, 5]
2 for number in numbers:
3     print(number)
```

Output:

```
1 1
2 2
3 3
4 4
5 5
```

- ii. Using a `while` Loop:

PYTHON

```
1 numbers = [1, 2, 3, 4, 5]
2 i = 0
3 while i < len(numbers):
4     print(numbers[i])
5     i += 1
```

Output:

```
1 1
2 2
3 3
4 4
5 5
```

18. What are dictionaries in Python?

Dictionaries store key-value pairs with unique keys. Access, add, or remove items using keys, which makes it efficient for lookups.

Dictionaries are defined using curly braces

`{}` with key-value pairs separated by commas and a colon `:` between each key and its corresponding value.

19. How do you add or remove elements from a dictionary?

Add or Remove elements (key-value pairs) from a dictionary using various methods and operations.

i. Adding Elements:

- **Using Assignment:** You can add a new key-value pair to a dictionary by assigning a value to a new key.

PYTHON

```
1 student_ages = {"Alice": 20, "Bob": 21}
2 student_ages["Charlie"] = 19
3 print(student_ages) # Output: {"Alice": 20, "Bob": 21, "Charlie": 19}
```

- Using the

`update()` Method: You can add multiple key-value pairs to a dictionary at once using the `update()` method.

PYTHON

```
1 student_ages = {"Alice": 20, "Bob": 21}
2 student_ages.update({"Charlie": 19, "David": 22})
3 print(student_ages) # Output: {"Alice": 20, "Bob": 21, "Charlie": 19, "David": 22}
```

ii. Removing Elements:

- Using the `del` Statement:

You can remove a key-value pair from a dictionary using the

`del` statement.

PYTHON

```
1 student_ages = {"Alice": 20, "Bob": 21, "Charlie": 19}
2 del student_ages["Bob"]
3 print(student_ages) # Output: {"Alice": 20, "Charlie": 19}
```

- Using the `pop()` Method:

You can remove a key-value pair from a dictionary and get the value of the removed key using the

`pop()` method.

PYTHON

```
1 student_ages = {"Alice": 20, "Bob": 21, "Charlie": 19}
2 removed_age = student_ages.pop("Bob")
3 print(removed_age) # Output: 21
4 print(student_ages) # Output: {"Alice": 20, "Charlie": 19}
```

- Using the `popitem()` Method:

You can remove and return an arbitrary key-value pair from a dictionary using the

`popitem()` method.

PYTHON

```
1 student_ages = {"Alice": 20, "Bob": 21, "Charlie": 19}
2 removed_item = student_ages.popitem()
3 print(removed_item) # Output: ("Charlie", 19)
4 print(student_ages) # Output: {"Alice": 20, "Bob": 21}
```

- Using the `clear()` Method:

You can remove all key-value pairs from a dictionary using the

`clear()` method.

PYTHON

```

1 student_ages = {"Alice": 20, "Bob": 21, "Charlie": 19}
2 student_ages.clear()
3 print(student_ages) # Output: {}

```

These methods provide flexibility for adding or removing elements from dictionaries in Python, allowing you to manipulate dictionary contents according to your requirements.

20. What are sets in Python?

In Python, a set is a built-in data type that represents an unordered collection of unique elements. Sets are used to store distinct items, and they do not allow duplicate elements. Sets are defined using curly braces

`{}` with elements separated by commas.

21. How do you perform set operations in Python?

Use operators like

`|` for union, `&` for intersection, `-` for difference, and `^` for symmetric difference.

i. Union (`|`):

Returns a set containing all unique elements present in either set.

PYTHON

```

1 set1 = {1, 2, 3}
2 set2 = {3, 4, 5}
3 union_set = set1 | set2
4 print(union_set) # Output: {1, 2, 3, 4, 5}

```

ii. Intersection (`&`):

Returns a set containing elements that are common to both sets.

PYTHON

```

1 set1 = {1, 2, 3}
2 set2 = {3, 4, 5}
3 intersection_set = set1 & set2
4 print(intersection_set) # Output: {3}

```

iii. Difference (`-`):

Returns a set containing elements that are present in the first set but not in the second set.

PYTHON

```

1 set1 = {1, 2, 3}
2 set2 = {3, 4, 5}
3 difference_set = set1 - set2
4 print(difference_set) # Output: {1, 2}

```

iv. Symmetric Difference (`^`):

Returns a set containing elements that are present in either set, but not in both sets.

PYTHON

```

1 set1 = {1, 2, 3}
2 set2 = {3, 4, 5}
3 symmetric_difference_set = set1 ^ set2
4 print(symmetric_difference_set) # Output: {1, 2, 4, 5}

```

v. Subset (`<=`):

Returns

True if all elements of one set are present in another set.

PYTHON

```
1 set1 = {1, 2}
2 set2 = {1, 2, 3, 4}
3 is_subset = set1 <= set2
4 print(is_subset) # Output: True
```

vi. Superset (

>=): Returns **True** if a set contains all elements of another set.

PYTHON

```
1 set1 = {1, 2, 3, 4}
2 set2 = {1, 2}
3 is_superset = set1 >= set2
4 print(is_superset) # Output: True
```

22. What are Python functions?

Function is a block of reusable code that performs a specific task. Functions provide a way to organize and modularize code by breaking it into smaller, manageable pieces. Functions encapsulate a set of statements that are executed when the function is called, and they can accept input arguments (parameters) and return output values.

PYTHON

```
1 def function_name(parameter1, parameter2, ...):
2     # Function body (statements)
3     # Perform operations using parameters
4     # Optionally, return a value
```

23. How do you define a function in Python?

Use the

def keyword followed by a function name and parameters in parentheses, then add the function body indented below.

PYTHON

```
1 def function_name(parameter1, parameter2, ...):
2     # Function body
3     # Code to be executed
```

24. What is a lambda function in Python?

A lambda function is a small anonymous function defined using

lambda keyword, capable of taking any number of arguments, but can only have one expression.

Example of a lambda function that adds two numbers

PYTHON

```
1 add = lambda x, y: x + y
2 print(add(2, 3)) # Output: 5
```

25. What are global and local variables in Python?

Global variables

- Global variables are defined outside of any function or class, making them accessible from anywhere within the program, including inside functions and classes.
- Global variables are accessible in the entire module in which they are defined, as well as in any other modules that import the module where the global variable is defined.
- Global variables can be accessed and modified from any part of the program.

Example

PYTHON

```

1  global_var = 10
2
3  def func():
4      print(global_var)
5
6  func() # Output: 10

```

Local Variables:

- Local variables are defined within a function or a code block (such as a loop or conditional statement), making them accessible only within that specific scope.
- Local variables have a limited scope and are not accessible from outside the function or code block where they are defined.
- Each function call creates a new scope, and variables defined within the function are local to that scope.

Example:

PYTHON

```

1  def func():
2      local_var = 20
3      print(local_var)
4
5  func() # Output: 20
6  # print(local_var) # This would raise a NameError because local_var is not defined outside func

```

26. How do you handle exceptions in Python?

Use

try-except blocks to catch and handle exceptions. Optionally, use **else** for code that runs if no exceptions, and **finally** for code that always runs.

The key components of exception handling in Python are:

- try-except Block:** The `try` block is used to wrap the code that may potentially raise an exception.
- except Block:** An `except` block follows a `try` block and specifies the type of exception it can handle.
- else Block (Optional):** An `else` block can optionally follow the `except` block(s).
- finally Block (Optional):** A `finally` block can optionally follow the `try` and `except` block(s).

Here's an example of exception handling in Python:

PYTHON

```

1  try:
2      x = 10 / 0 # This will raise a ZeroDivisionError
3  except ZeroDivisionError:
4      print("Error: Division by zero")
5  else:
6      print("Division successful")
7  finally:
8      print("Cleanup code executed")

```

27. What are Python modules?

Module is a file containing Python code that defines variables, functions, and classes. Modules allow you to organize your code logically into reusable units, making it easier to manage and maintain large codebases.

Here are some key points about Python modules:

- i. Organization
- ii. Code Reusability
- iii. Namespaces
- iv. Importing Modules
- v. Standard Library
- vi. Third-Party Modules

28. How do you import a module in Python?

Import modules using the

`import` statement. There are several ways to import modules, depending on how you want to access the module's contents and how you want to refer to them in your code.

i. Importing the Entire Module:

Syntax:

```
1 import module_name
```

PYTHON

Example:

```
1 import math
2 print(math.pi) # Output: 3.141592653589793
```

PYTHON

ii. Importing Specific Objects from a Module:

Syntax:

```
1 from module_name import object1, object2, ...
```

PYTHON

Example:

```
1 from math import pi, sqrt
2 print(pi) # Output: 3.141592653589793
3 print(sqrt(4)) # Output: 2.0
```

PYTHON

iii. Importing a Module with an Alias:

Syntax:

```
1 import module_name as alias
```

PYTHON

Example:

PYTHON

```
1 import math as m
2 print(m.pi) # Output: 3.141592653589793
```

iv. Importing All Objects from a Module:

Syntax:

PYTHON

```
1 from module_name import *
```

Example:

PYTHON

```
1 from math import *
2 print(pi) # Output: 3.141592653589793
```

29. What is the

if __name__ == "__main__" statement?

The

if __name__ == "__main__" statement in Python is used to determine whether a script is being run as the main program or being imported as a module into another script. This allows you to control the execution of code blocks depending on the context in which the script is run.

Explanation

- **__name__** : A special built-in variable in Python. It is set to **"__main__"** when the script is executed directly. If the script is imported as a module, **__name__** is set to the module's name.
- **if __name__ == "__main__"** : This condition evaluates to **True** if the script is run directly, allowing the code block under it to execute. If the script is imported as a module, the code block is not executed.

30. How do you create a class in Python?

Use the

class keyword followed by the class name and a colon, then define class attributes and methods indented inside.

Here's an example of a simple class definition in Python:

PYTHON

```
1 class Person:
2     # Class attribute
3     species = "Human"
4
5     # Constructor method (initializer)
6     def __init__(self, name, age):
7         # Instance attributes
8         self.name = name
9         self.age = age
10
```

Expand

31. How do you create a child class in Python?

Define a child class by extending the parent class in the class declaration using the syntax:

```
class ChildClass(ParentClass):
```

Here's an example of creating a child class in Python:

```
1 class Animal:
2     def speak(self):
3         return "Animal speaks"
4
5 class Dog(Animal):
6     def bark(self):
7         return "Woof!"
8
9 class Cat(Animal):
10    def meow(self):
```

PYTHON

Expand

32. What are Python decorators?

Python decorators are a powerful feature that allows you to modify or extend the behavior of functions or methods without changing their source code. Decorators are essentially functions themselves that wrap around other functions or methods to add additional functionality or behavior to them dynamically.

33. How do you read and write files in Python?

Read and write files using built-in functions and methods provided by the

`open()` function and file objects. Here's how you can read and write files in Python:

i. Opening a File: To open a file, you use the `open()` function, specifying the file path and the mode in which you want to open the file. The most common modes are:

- 'r' : Read mode (default). Opens the file for reading.
- 'w' : Write mode. Opens the file for writing. If the file exists, it will be overwritten. If the file doesn't exist, a new file will be created.
- 'a' : Append mode. Opens the file for writing. If the file exists, new data will be appended to the end of the file. If the file doesn't exist, a new file will be created.
- 'b' : Binary mode. Opens the file in binary mode (e.g., for reading or writing binary data).
- 't' : Text mode (default). Opens the file in text mode (e.g., for reading or writing text data).

Example:

```
1 # Open a file in read mode
2 file = open("example.txt", "r")
3
4 # Open a file in write mode
5 file = open("example.txt", "w")
6
7 # Open a file in append mode
8 file = open("example.txt", "a")
```

PYTHON

ii. Reading from a File:

Once you've opened a file for reading, you can use methods like

`read()` , `readline()` , or `readlines()` to read data from the file.

Example:

```
1 # Open a file in read mode
2 file = open("example.txt", "r")
3
4 # Read the entire contents of the file
5 content = file.read()
6 print(content)
7
8 # Close the file
```

```
8 # Close the file
9 file.close()
```

iii. Writing to a File:

When a file is opened in write mode, you can use methods like

`write()` or `writelines()` to write data to the file.

Example:

PYTHON

```
1 # Open a file in write mode
2 file = open("example.txt", "w")
3
4 # Write data to the file
5 file.write("Hello, world!\n")
6 file.write("This is a test.\n")
7
8 # Close the file
9 file.close()
```

iv. Closing a File:

After you're done reading from or writing to a file, it's important to close the file using the

`close()` method.

Example:

PYTHON

```
1 file.close()
```

v. Using `with` Statement (Context Manager):

Python's

`with` statement can be used as a context manager to automatically close the file when the block of code inside the `with` statement is exited.

Example:

PYTHON

```
1 with open("example.txt", "r") as file:
2     content = file.read()
3     print(content)
```

34. What are Python iterators?

Iterator is an object that represents a stream of data or a sequence of elements. Iterators provide a way to access elements of a collection one at a time without needing to know the underlying data structure or implementation details.

The iterator protocol in Python defines two methods that iterators must implement:

- i. `__iter__()` : This method returns the iterator object itself. It is called when the `iter()` function is called on the iterable object.
- ii. `__next__()` : This method returns the next element from the iterator. It is called repeatedly to retrieve elements from the iterator, and it raises a `StopIteration` exception when there are no more elements to be retrieved.

35. What is recursion in Python and how do you use it?

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a problem. In Python, recursion is a powerful and elegant way to solve problems that can be broken down into smaller, similar subproblems. It often leads to concise and readable code, especially for tasks involving tree structures, backtracking, and divide-and-conquer algorithms.

The key components of recursion are:

- i. Base Case: A terminating condition that defines when the recursion should stop. It prevents the function from infinitely calling itself and ensures that the recursion eventually reaches a stopping point.
- ii. Recursive Case: A condition that calls the function recursively with a modified input, moving closer to the base case. It allows the function to break down the problem into smaller subproblems until it reaches the base case.

Here's an example of a simple recursive function in Python that calculates the factorial of a non-negative integer:

PYTHON

```
1 def factorial(n):
2     # Base case: factorial of 0 or 1 is 1
3     if n == 0 or n == 1:
4         return 1
5     # Recursive case: factorial of n is n * factorial(n - 1)
6     else:
7         return n * factorial(n - 1)
8
9 # Example usage
10 print(factorial(5)) # Output: 120
```

[Submit Fe](#)